

// Jivops

Guía completa — 30 lecciones

Arquitectura · Métricas · Logs · Trazas y OpenTelemetry · Operación a
escala · Producción avanzada

Contenido

Módulo 1: Fundamentos y arquitectura

- Lección 01 — Qué es Grafana Alloy y su rol como colector unificado
- Lección 02 — Arquitectura de componentes — el modelo de pipeline de Alloy
- Lección 03 — Alloy configuration language (River/HCL-like syntax)
- Lección 04 — Instalación de Alloy (binario, Helm, Docker)
- Lección 05 — Alloy vs Grafana Agent vs OpenTelemetry Collector

Módulo 2: Recolección de métricas

- Lección 06 — Componentes prometheus.scrape y discovery
- Lección 07 — Relabeling en Alloy
- Lección 08 — Remote write hacia Prometheus/Mimir
- Lección 09 — Recolección de métricas de Kubernetes con Alloy
- Lección 10 — Componentes de procesamiento (prometheus.relabel, prometheus.remote_write)

Módulo 3: Recolección de logs

- Lección 11 — Componentes loki.source — recolección de logs
- Lección 12 — Pipelines de procesamiento de logs (loki.process)
- Lección 13 — Envío de logs hacia Loki (loki.write)
- Lección 14 — Migración de configuración de Promtail a Alloy
- Lección 15 — Recolección de logs de Kubernetes con Alloy

Módulo 4: Trazas y OpenTelemetry

- Lección 16 — Componentes OTel — otelcol.receiver, otelcol.processor
- Lección 17 — Recolección de trazas distribuidas con Alloy
- Lección 18 — Exportación hacia Tempo
- Lección 19 — Correlación de logs, métricas y trazas (LGTM stack)
- Lección 20 — Alloy como Collector OpenTelemetry nativo

Módulo 5: Operación a escala

- Lección 21 — Clustering de Alloy — modo distribuido
- Lección 22 — Escalado horizontal y balanceo de carga de recolección
- Lección 23 — Alta disponibilidad de Alloy
- Lección 24 — Monitoreo del propio Alloy (self-monitoring)
- Lección 25 — Troubleshooting de pipelines de Alloy

Módulo 6: Producción avanzada

Lección 26 — Gestión de secretos en configuración de Alloy

Lección 27 — Despliegue de Alloy con GitOps

Lección 28 — Migración completa desde stack legacy a Alloy

Lección 29 — Rendimiento y tuning de componentes

Lección 30 — Proyecto final: pipeline de observabilidad completo con Grafana Alloy

Módulo 1: Fundamentos y arquitectura

Lección 01 — Qué es Grafana Alloy y su rol como colector unificado

Alloy es el sucesor de Grafana Agent, unificando métricas, logs y trazas en un único binario, reduciendo los agentes distintos a mantener.

Lección 02 — Arquitectura de componentes — el modelo de pipeline de Alloy

Los componentes encadenados permiten combinar piezas reutilizables (scrape, relabel, write) de forma modular, adaptando el pipeline a cada necesidad.

Lección 03 — Alloy configuration language (River/HCL-like syntax)

Referenciar directamente la salida de un componente como entrada de otro evita repetir configuración, siendo más expresivo que un YAML estático.

```
targets = discovery.kubernetes.pods.targets
```

Lección 04 — Instalación de Alloy (binario, Helm, Docker)

Desplegar Alloy como DaemonSet garantiza que cada nodo tenga su propia instancia recolectando telemetría local, sin depender de una única central.

```
helm install alloy grafana/alloy
```

Lección 05 — Alloy vs Grafana Agent vs OpenTelemetry Collector

Alloy incorpora componentes otelcol compatibles con OpenTelemetry, permitiendo reutilizar configuración existente sin reescribirla desde cero.

Módulo 2: Recolección de métricas

Lección 06 — Componentes prometheus.scrape y discovery

Separar discovery de scrape permite reutilizar la misma salida de descubrimiento en múltiples pipelines distintos, sin duplicar la lógica.

```
targets = discovery.kubernetes.pods.targets
```

Lección 07 — Relabeling en Alloy

Descartar métricas de bajo valor mediante relabeling antes de enviarlas evita pagar el costo de ingesta y almacenamiento de datos que nunca se consultan.

Lección 08 — Remote write hacia Prometheus/Mimir

Configurar múltiples endpoints permite enviar las mismas métricas a dos backends simultáneamente, útil durante una migración gradual.

```
url = "http://mimir:9009/api/v1/push"
```

Lección 09 — Recolección de métricas de Kubernetes con Alloy

Combinar métricas de aplicación, kube-state-metrics y cAdvisor da una imagen completa del cluster en distintos niveles combinados.

Lección 10 — Componentes de procesamiento (prometheus.relabel, prometheus.remote_write)

El flujo lógico es discovery → scrape → relabel → remote_write: los datos deben transformarse antes de enviarse al destino final.

Módulo 3: Recolección de logs

Lección 11 — Componentes loki.source — recolección de logs

loki.source.kubernetes cumple el rol que antes tenía Promtail como herramienta separada, ahora integrado como un componente más de Alloy.

Lección 12 — Pipelines de procesamiento de logs (loki.process)

loki.process mantiene un modelo conceptual similar a los stages de Promtail, facilitando la migración de configuraciones ya existentes.

```
stage.json { expressions = { level = "level" } }
```

Lección 13 — Envío de logs hacia Loki (loki.write)

Al estar desacoplado del resto, se puede cambiar el destino sin modificar cómo se recolectan y procesan los logs previamente.

```
url = "http://loki:3100/loki/api/v1/push"
```

Lección 14 — Migración de configuración de Promtail a Alloy

El comando 'alloy convert' automatiza la traducción de configuración existente, reduciendo el riesgo de errores manuales al reescribir todo.

```
alloy convert --source-format=promtail
```

Lección 15 — Recolección de logs de Kubernetes con Alloy

Reutilizar el mismo discovery.kubernetes para métricas y logs mantiene labels consistentes entre ambos tipos de telemetría del mismo Pod.

Módulo 4: Trazas y OpenTelemetry

Lección 16 — Componentes OTel — `otelcol.receiver`, `otelcol.processor`

Recibir datos vía el protocolo estándar OTLP hace a Alloy compatible con cualquier aplicación instrumentada con un SDK oficial de OpenTelemetry.

```
endpoint = "0.0.0.0:4317"
```

Lección 17 — Recolección de trazas distribuidas con Alloy

A diferencia de los logs pasivos, las trazas requieren que la aplicación genere activamente spans usando un SDK de OpenTelemetry.

Lección 18 — Exportación hacia Tempo

Cada tipo de telemetría usa un protocolo y exporter especializado: `otelcol.exporter.otlp` para trazas, distinto de `loki.write` o `remote_write`.

```
endpoint = "tempo:4317"
```

Lección 19 — Correlación de logs, métricas y trazas (LGTM stack)

Un identificador común como `trace_id` presente en logs y trazas permite saltar desde una traza específica directamente a los logs de esa petición.

Lección 20 — Alloy como Collector OpenTelemetry nativo

Combinar componentes nativos de Prometheus/Loki con componentes OTel da flexibilidad a organizaciones con fuentes de datos heterogéneas.

Módulo 5: Operación a escala

Lección 21 — Clustering de Alloy — modo distribuido

Sin clustering, múltiples instancias escaneando los mismos targets generarían datos duplicados y carga de scraping innecesaria.

```
cluster { enabled = true }
```

Lección 22 — Escalado horizontal y balanceo de carga de recolección

El hashing consistente minimiza la reasignación de targets ante un cambio, redistribuyendo solo una porción pequeña en vez de todo.

Lección 23 — Alta disponibilidad de Alloy

Si una réplica falla, el clustering redistribuye automáticamente sus targets hacia las restantes, evitando un hueco prolongado en la recolección.

```
kubectl scale deployment alloy --replicas=3
```

Lección 24 — Monitoreo del propio Alloy (self-monitoring)

Sin monitorear las métricas internas de Alloy, un fallo de envío hacia el backend podría pasar desapercibido hasta faltar datos críticos.

Lección 25 — Troubleshooting de pipelines de Alloy

La UI de depuración integrada muestra el grafo de componentes y su estado, más rápido de interpretar que logs de texto plano dispersos.

Módulo 6: Producción avanzada

Lección 26 — Gestión de secretos en configuración de Alloy

Usar una variable de entorno para la credencial evita que quede expuesta en el historial de Git del archivo de configuración versionado.

```
password = env("REMOTE_WRITE_PASSWORD")
```

Lección 27 — Despliegue de Alloy con GitOps

Tener el historial de cambios en Git permite rastrear exactamente qué commit modificó el scraping y quién lo hizo, facilitando el diagnóstico.

Lección 28 — Migración completa desde stack legacy a Alloy

Migrar primero un solo tipo de telemetría reduce el riesgo de la primera migración, permitiendo ajustar el proceso antes de aplicarlo al resto.

Lección 29 — Rendimiento y tuning de componentes

Aumentar el `scrape_interval` reduce la carga total sobre los targets, a costa de menor resolución temporal en los datos recolectados.

```
scrape_interval = "30s"
```

Lección 30 — Proyecto final: pipeline de observabilidad completo con Grafana Alloy

Se integra todo lo anterior: recolección unificada de métricas/logs/trazas, clustering para HA, gestión segura de secretos, despliegue GitOps, self-monitoring y tuning de rendimiento — el nivel esperado de un pipeline de observabilidad con Alloy listo para producción real.